

# Your Agent is Underperforming: The Case for Agent Performance Improvement Plans

Anonymous Submission

*Author information withheld per review guidelines*

**Abstract**—Production AI agents frequently degrade after deployment, yet structured remediation culture is largely absent. We introduce the Agent Performance Improvement Plan (APIP): a lifecycle protocol adapted from organizational management that governs the remediation of underperforming agents through statistically disciplined triggers on a behavioral contract, measurable cause attribution over a five-category failure-mode taxonomy, tiered interventions, and a bounded window with formal exit criteria. We evaluate the protocol end-to-end in three live multi-agent environments—customer service ( $\tau^2$ -bench), software engineering (ChatDev), and economic operations (CoffeeBench)—injecting each failure mode with known ground truth. APIP-governed remediation achieves [X%] of oracle time-to-resolution versus [Y%] for random-tier selection, and attribution identifies the injected failure mode with accuracy [Z] while abstaining honestly in [W%] of cases—to our knowledge the first attribution confusion matrix reported for a remediation protocol. A declarative schema meets AI governance traceability requirements, and we demonstrate and attenuate the mesh disruption failure mode, where a locally optimal new agent degrades peer performance.

**Index Terms**—AI agents, post-deployment monitoring, remediation, behavioral drift, AI governance, multi-agent systems, performance improvement plan

## I. INTRODUCTION

Organizations increasingly deploy conversational agents as their main customer interfaces, handling support tickets, returns, and billing disputes at scale. Pre-release evaluation is well-equipped—standardized benchmarks [1], red-teaming [2], RLHF alignment [3]—but there is a notable lack of culture around post-deployment remediation: a comprehensive study of deployed agents found many fail or underperform, with limited understanding of how organizations react [4].

Ad-hoc responses to degraded agents are standard: engineers “fix” individual prompts, product managers file tickets, systems get rolled back—with no structure, no documentation, and no evaluation of whether the fixes worked. Agents recover through informal means or are quietly removed. Beyond the operational cost, this is a governance risk: the EU AI Act requires documentation and human oversight of deployed AI systems [5], and Wachter et al. [6] note that accountability in automated decision-making requires traceability, which formalized remediation can provide.

We recommend the performance improvement plan (PIP) as the model: an established HR tool containing the elements common to accountability protocols—a trigger, cause identification, a bounded corrective window, check-ins, and an ex-

plicit definition of recovery or exit. Kirkpatrick’s [7] four-level model—reaction, learning, behavior, results—complements it by evaluating whether remediation produced lasting change.

We make the following contributions:

- We introduce the APIP, a formalized post-deployment remediation lifecycle protocol grounded in organizational management and training evaluation research.
- We develop a five-category failure-mode taxonomy with a published deterministic mapping from trace-level annotations (MAST [8]) and disaggregation signatures onto taxonomy categories and intervention tiers.
- We propose a three-tier intervention hierarchy with escalation criteria, evaluated through Kirkpatrick’s framework, and a declarative APIP schema for tooling and audit trails.
- We evaluate the full protocol in three live multi-agent environments against a four-condition baseline family, and demonstrate the mesh disruption problem together with a scoped-context onboarding mitigation.

## II. BACKGROUND AND RELATED WORK

### A. Agent Evaluation and the Deployment Gap

Pre-deployment evaluation has matured—benchmarks [1], red-teaming [2], RLHF [3]—while post-deployment monitoring has received far less systematic attention. Pan, Arabzadeh, and Ellis [4] conducted the first large-scale study of production agents, finding most deployments fail or underdeliver with little known about post-release management; AgentCompass [9] addresses detection but prescribes no remediation; Rath [10] projects a 42% task-success reduction for unchecked drift. On the diagnosis side, MAST [8] provides an empirically derived failure taxonomy with an LLM-annotator pipeline; it classifies *what went wrong in a trace* but does not select a remediation, and we adopt it as the trace-labeling stage of APIP cause attribution (Section IV-B). The APIP addresses what these lines of work leave open: a structured remediation protocol once degradation is identified.

Run-level debugging toolkits such as AgentDebugX [11] close a detection–attribution–recovery–rerun loop for a single failed execution; the APIP operates one level above, deciding whether an aggregate degradation warrants intervention, at what tier, and until when. A successful rerun is Level 1/2 evidence (Section II-C), not durable recovery against the behavioral contract.

### B. The Performance Improvement Plan as a Structural Template

The PIP in organizational management is a formal document specifying the deficiencies observed, root causes identified, corrective actions required, a time window, a check-in schedule, and success criteria [12]. The analogy is productive but requires qualification: a human employee can self-direct improvement; a deployed agent cannot. The APIP is therefore imposed entirely by human operators—the agent is the subject, not the author, of the plan—collapsing the manager–employee dyad into the operator, who both writes the plan and implements it.

### C. Kirkpatrick’s Four-Level Evaluation Framework

Kirkpatrick’s four-level model [7], [12] evaluates interventions hierarchically—Level 1 (Reaction), Level 2 (Learning), Level 3 (Behavior: transfer to on-the-job change), Level 4 (Results: organizational outcomes)—and we adopt it as the template for evaluating APIP interventions: did the intervention change immediate output patterns (Level 1/2), produce durable behavioral change on the failure-mode class (Level 3), and improve the contract’s business metrics (Level 4)? An intervention that improves Level 1/2 metrics without Level 3/4 improvement has not remediated the agent; it has merely suppressed the symptom.

### D. AI Governance and Documentation Requirements

Regulatory frameworks increasingly require documentation, oversight, and traceability: the EU AI Act [5] mandates post-market monitoring and logs sufficient to identify sources of error, and Kolt, Heim, and Anderljung [13] argue accountability requires linking actions to specific agents and operators via identifiers, real-time monitoring, and activity logging. The APIP schema’s `apip_id`, `agent_id`, `owner`, and `exit_evaluated_at` fields operationalize this linkage, positioning the APIP as a governance instrument.

## III. A TAXONOMY OF AGENT FAILURE MODES

A critical prerequisite to structured remediation is structured cause-finding. We propose a five-category taxonomy for customer-facing agents (Table I): four intrinsic modes—behavioral drift, input brittleness, alignment creep, tool misuse—plus one exogenous mode, mesh disruption (`exogenous_disruption`), in which the failing agent is not the cause of its own degradation. The categories suggest distinct remediation pathways—the operationally important property—each corresponding to a different level of the agent’s behavioral stack and consequently a different intervention tier.

Two properties deserve emphasis. Input brittleness is masked by aggregate metrics—an agent with 85% overall success may fail catastrophically on the 8% of requests from one segment—so the trigger must monitor disaggregated splits. And behavioral drift is a freshness failure remediated by retrieval refresh, not fine-tuning, while alignment creep is value misalignment, distinct from capability failure [19].

TABLE I  
AGENT FAILURE MODE TAXONOMY WITH KIRKPATRICK LEVEL MAPPING AND REMEDIATION TIER.

| Failure Mode         | Description & Signals                                                                                                                                                                                | Remediation Tier                                   |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Behavioral Drift     | Gradual degradation as world-state diverges from training or retrieval context. Signals: rising hallucination rate on new entities, outdated policy citations. Kirkpatrick Level 4 (Results).        | Tier 2: Retrieval / RAG update                     |
| Input Brittleness    | Strong on modal inputs, catastrophic on a specific subclass. Signals: bimodal CSAT, failure clustering by segment or topic. Level 2 (Learning/Capability).                                           | Tier 1 (examples) or Tier 3 (fine-tune)            |
| Alignment Creep      | Subtle drift in tone, value expression, or policy compliance. Signals: rising review flags, brand guideline violations. Level 3 (Behavior).                                                          | Tier 1: Prompt constraint injection                |
| Tool Misuse          | Correct intent, incorrect tool invocation. Signals: action errors, silent downstream failures. Level 3 (Behavior).                                                                                   | Tier 1 (prompt) or Tier 3 (fine-tune)              |
| Exogenous Disruption | Peer agents degrade after a mesh change event; no intrinsic mode found in them. Signals: degradation concentrated in peer roles, onset at a mesh-composition event. Level 4 (Results), system level. | Mesh protocol (Sec. VI-A), not an agent-level tier |

**Exogenous Disruption** is the taxonomy’s escape hatch: cause attribution correctly concludes that *no* intrinsic mode explains the degradation, because the cause lies in a mesh change—most prominently the introduction of a new, individually competent peer agent. Its signature is structural rather than behavioral (degradation concentrated in peer roles, coincident with a mesh-composition event, the degraded agents’ own traces annotating clean), and its remediation is a mesh-level protocol rather than an agent-level tier (Section VI-A).

### A. From Trace Labels to Taxonomy Categories

The taxonomy is operational only if a degradation window can be classified by procedure rather than intuition. Cause attribution combines a trace-level label distribution from MAST [8] with a *disaggregation signature*—how degradation distributes across task categories, agent roles, and time—via a published deterministic mapping (supplementary material); two mapping rows share a MAST cluster but differ on signature, which is why the taxonomy is two-level. The mapping is frozen before any evaluated run; below-confidence combinations *abstain*, logged and scored as attribution failures rather than silently resolved.

A further category, **Adversarial Subversion**, captures failures whose proximate cause is an adaptive attack rather than capability or value drift: frontier models can embed prompt injections that evade diverse monitors, and resampling-based protocols can amplify them [15], [16]. We treat it as separate and out of scope because prompt revision, retrieval refresh, and fine-tuning address symptoms, not the adversarial dynamic; effective response—monitor diversification, defer-to-trusted protocols, reduced action authority—is drawn from the AI Control literature [14], which operates at a complementary granularity: per-action runtime decisions under an adversarial

threat model, versus the APIP’s aggregated behavioral-contract window under presumed capability or value drift. Control-protocol audit decisions can feed APIP triggers, adversarial attribution escalates out of APIP’s tier model, and the schema accommodates the category through the `adversarial_subversion_failure_mode` value.

#### IV. THE APIP FRAMEWORK

We now describe the APIP as a structured lifecycle protocol of five phases: Trigger, Cause Attribution, Intervention Planning, Remediation Window, and Exit Evaluation.

##### A. Phase 1: Trigger

An APIP initiates when performance crosses a trigger threshold on monitored metrics. This presupposes a *behavioral contract*—a formal specification, written at deployment time, of task surface, success metrics and measurement methodology, acceptable ranges, escalation policy, and human review points; most teams currently skip it, delegating expectations to informal organizational memory. Two trigger modalities are both necessary: absolute floors catch severe degradation, while relative drift triggers catch gradual decline that may never breach a floor.

1) *Statistical discipline of the trigger*: Implementing the trigger phase against live agents exposed a failure mode of the naive design that we consider a finding in its own right. Thresholds must be *derived*, not hand-picked: each contract metric’s envelope is estimated from a calibration window as  $\text{mean} \pm k\sigma$  (with  $k$  reported), which preempts the objection that thresholds were chosen so that triggers would fire where desired. Even then, a single-gate rule—compare a rolling mean against the derived floor, across the 15–25 metrics and disaggregated splits a realistic contract monitors—breached in 75% of *clean, zero-injection* runs, because a small calibration sample is treated as exact and the per-tick tests are uncorrected. A breach must therefore pass two gates: the degradation must be at least the contract’s  $k\sigma$  (it matters), and it must survive Benjamini–Hochberg adjustment across the metric family against the correct null (it is real). Disaggregated groups additionally require a minimum per-group episode count (a power floor); under-powered groups receive no envelope, and the exclusion is recorded rather than silent. Finally, the detector’s significance level is swept on clean-arm runs only and then frozen before any evaluated window—clean data carries no information about where triggers *should* fire, so this calibration does not compromise pre-registration. Together these changes reduced clean-run breaches to 30% and group-level false alarms from 71 to 0 per 700 ticks in our harness (Section V). A trigger phase without this discipline initiates APIPs on noise, and every downstream phase inherits the error.

##### B. Phase 2: Cause Attribution

Once triggered, the APIP requires a structured cause attribution process before any intervention is applied. The goal is to identify which failure mode category (Section III) best explains the degradation. Tier mismatch from misdiagnosis

is the primary source of avoidable regression: prompt engineering applied to a retrieval freshness problem, for instance, consumes engineering cycles with no improvement.

This is distinct from, and composes with, automated failure attribution, which localizes the responsible agent or step within a single failed trajectory [20]–[22]. APIP Phase 2 operates one level of aggregation above, classifying a window-level degradation into a remediation-oriented category to select a tier; trajectory-level outputs are natural evidence inputs, but localizing the failing step falls short of diagnosing the underlying cause—the gap the mandatory taxonomy classification closes at the deployment level.

The process should include disaggregated performance analysis to isolate failure locus; human-adjudicated analysis of sampled failure cases, applying automated trajectory-level attribution where tooling permits [11], [20], [22]; retrieval audit if drift is suspected; tool invocation log analysis if misuse is suspected; and structured classification into the taxonomy, closing with a written cause attribution report that becomes part of the APIP record.

1) *A measurable instantiation: the three-stage funnel*: We instantiate this as a three-stage funnel scoreable against ground truth: (1) every contract metric is split by task category, agent role, and user segment, producing a disaggregation signature; (2) a MAST annotator [8] labels a sample of failing traces, with the judge required to differ from the agent’s backbone—preferably by model family—to avoid a judge sharing the failure modes it must detect; (3) the frozen mapping (Section III-A) combines signature and label distribution into a `failure_mode` with a confidence score, returning `abstain` below threshold. The mapping table is hashed into every run manifest, making post-hoc edits detectable, and the funnel’s discrete output is scored as a confusion matrix against injected ground truth (Section V), with abstention rate reported alongside—never instead of—accuracy.

##### C. Phase 3: Intervention Planning and the Tier Model

Interventions are organized into three tiers of increasing cost, disruption, and lead time; cause attribution selects the starting tier, with escalation permitted within the window. **Tier 1 (prompt-level)**: constraint injection, persona adjustment, example replacement—fastest, least disruptive, easiest to roll back; the first response to alignment creep and much input brittleness. **Tier 2 (retrieval and context)**: RAG index updates, knowledge-base revisions, tool schema fixes—addressing drift from world-state divergence and tool misuse from stale descriptions, with contract validation before re-deployment. **Tier 3 (model-level)**: fine-tuning, DPO, or replacement—highest cost and regression risk, warranted when lower tiers are exhausted or the failure mode is embedded in model weights.

##### D. Phase 4: Remediation Window and Check-In Cadence

The remediation window bounds the period during which interventions are implemented and evaluated, creating urgency and a structured decision point. Duration is calibrated to

the tier—7–14 days for Tier 1, 30–60 for Tier 3—mirroring organizational PIP durations of 30–90 days [12].

Check-ins review performance against the recovery trajectory at multiple Kirkpatrick levels—flagging interventions that improve surface metrics without Level 3/4 change as potentially superficial—and act as early warnings: no improvement at the first check-in triggers tier escalation rather than waiting for expiry. A recommended cadence is 25%, 50%, and 75% of elapsed time.

#### E. Phase 5: Exit Evaluation

At window close, a formal exit evaluation records one of five outcomes: *resolved* (all contract metrics returned to envelope); *regressed* (envelope restored but breached again—the signature of a tier-mismatched intervention that suppressed symptoms); *timeout* (window expired without durable recovery; more fundamental intervention warranted); *retired* (the agent cannot be returned to contract at acceptable cost); and *recalibrated* (attribution concluded the *environment* legitimately changed—a moved goalpost—so the contract is re-baselined rather than the agent treated). We deliberately do not score recalibration as a resolution: conflating the two would let re-baselining masquerade as remediation success. Window *extension* is an in-protocol check-in decision, not an exit outcome, so every APIP terminates in exactly one of the five states; the record becomes part of the organizational AI audit trail.

#### F. The APIP Record as a Declarative Schema

Each APIP is recorded as an instance of a declarative schema (full field specification in supplementary material). Four field groups do architectural work: `behavioral_contract_ref` enforces writing a contract at deployment time, without which trigger and exit cannot be operationalized; `kirkpatrick_levels` captures multi-level check-in data, enabling post-hoc analysis of superficial versus durable improvement; `mapping_table_hash` and `environment_provenance` extend the visibility mechanisms of Kolt et al. [13] to *decision-rule* linkage—an auditor can verify the attribution rules and environment versions actually in force, converting pre-registration from assertion to checkable property; and `mesh_context` (Section VI-A) supports multi-agent cause attribution.

### V. EVALUATION IN THREE LIVE MULTI-AGENT ENVIRONMENTS

We evaluate the APIP end-to-end—calibration, trigger, attribution, tier selection, bounded window, exit—in three live multi-agent environments with real LLM inference and controlled failure-mode injection against known ground truth. Each environment supports a claim the others cannot (Table II); that the same contract/trigger/tier/exit machinery governs three unrelated domains is itself the generality argument for the behavioral contract abstraction.

TABLE II  
THE THREE EVALUATION ENVIRONMENTS AND THE CLAIM EACH SUPPORTS.

| Environment                    | What it buys the evaluation                                                                                                                                                                          |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\tau^2$ -bench (banking) [23] | Customer-service domain match to the deployments that motivate this work; deterministic database-state ground truth; a retrieval corpus that can be made stale independently of task ground truth.   |
| ChatDev [24]                   | Role-differentiated collaborative mesh (CEO → CTO → Programmer → Reviewer → Tester) with shared context—the substrate for alignment creep, disaggregated brittleness detection, and mesh disruption. |
| CoffeeBench [25]               | Persistent simulated time and a world that evolves without injection: <i>naturally occurring</i> degradation, answering the objection that injected drift merely mirrors its own schedule.           |

#### A. Shared Harness and Experimental Design

All three environments plug into a common controller through an adapter interface; traces are normalized to a shared span format, so the attribution pipeline is written once and reused unmodified. Each run proceeds in ticks: a calibration window derives every contract threshold as  $\mu \pm k\sigma$  (Section IV-A); failure modes are injected on a pre-registered schedule; both trigger modalities evaluate at every tick; breaches pass through the attribution funnel; and the selected intervention runs in a bounded window with check-ins at 25/50/75%, closing in one of the five exit outcomes.

**Baseline family.** Every injected cell runs under five conditions: *APIP* (the full protocol); *B1 no-action* (degradation runs to horizon); *B2 always-Tier-1* (a scripted prompt patch regardless of failure mode—the ad-hoc pattern typical of production practice, now one ablation); *B3 random-tier*; and *B4 oracle* (ground-truth mode revealed at trigger, correct tier applied immediately). The oracle bounds achievable performance, making the headline claim positional: APIP achieves [X%] of oracle time-to-resolution versus [Y%] for random-tier selection. Injected ground truth is quarantined from the controller—written into the run record for scoring and revealed only to B4; a test enforces that the APIP condition never branches on it.

**Statistical discipline.** Injection schedules, post-calibration thresholds, the frozen mapping table, and the analysis plan are fixed before evaluated runs; judge models are independent of backbone models by construction. We report bootstrap confidence intervals throughout, and note that variance across live environments substantially exceeds what parameterized simulation suggests—reporting that variance is part of the contribution.

#### B. $\tau^2$ -bench: Behavioral Drift with Deterministic Ground Truth

The primary drift experiment injects nothing into the agent; instead the *world* moves. Bank policy clauses and task ground truth advance on a deepening schedule (one additional clause every five ticks) while the agent’s retrieval corpus stays frozen—operationalizing the most familiar production drift

incident, an agent confidently citing superseded policy because its knowledge base was never re-indexed, with the deterministic database scoring every action objectively.

Detection fires after  $[n]$  ticks (judge-scored policy citation error rate; judge-human agreement  $[\kappa]$  on a verified episode sample). Under APIP, attribution returns `behavioral_drift` and the matched Tier 2 corpus refresh resolves in  $[a \pm b]$  ticks. Under B2, the Tier 1 prompt patch names the amended clauses known at patch time, recovers briefly, and regresses in  $[R\%]$  of runs as the ladder amends further clauses (TTR  $[c \pm d]$ ).

**Pilot observations.** Five single-seed pilots (7B backbone; 8B judge from a different family) preceded the pre-registered cells. Their central finding concerns remediation *completeness*: a first Tier 2 implementation closed the ticket without rebuilding the index, and the contract re-breached four ticks later; two further iterations re-breached on refreshes that were real but incomplete. Because exit evaluation re-measures the envelope rather than trusting the intervention log, the contract verifies that a remediation was *material and complete*, not merely performed—we conjecture such completeness is a dominant failure mode of real-world Tier 2 interventions. The pilots licensed one pre-freeze mapping-rule adjustment and produced two honest abstentions, scored as attribution failures under our own accounting.

#### C. ChatDev: Creep, Brittleness, and Mesh Disruption

**Alignment creep** perturbs the Reviewer persona cumulatively (“keep reviews brief”  $\rightarrow$  “avoid blocking progress”  $\rightarrow$  “approve unless catastrophic”). The Reviewer’s own flag rate *improves* while downstream test pass rate decays—the agent whose role is catching failures is the one degrading, and no absolute floor is crossed. Detection is possible only through the drift modality (tick  $[t]$ ), and the signature (no dominant trace-level failure mode; falling review-flag rate) maps to `alignment_creep` and a Tier 1 charter re-injection.

**Input brittleness** strips the Programmer’s GUI exemplars: GUI-category success collapses while aggregate success dips only modestly. Aggregate-only monitoring detects the failure after  $[n_1]$  ticks; disaggregated contract monitoring detects it after  $[n_2]$ . The gap is the direct, measured form of the masking argument in Section III, with the caveat that disaggregated envelopes require per-group power floors (Section IV-A) to avoid trading masked failures for false alarms.

**Mesh disruption** inserts a Documentation Specialist agent between Programmer and Reviewer at the injection tick. The new agent performs its own task well by judge assessment, yet peer metrics degrade by  $[\Delta]$  as it consumes shared context budget and shifts the input distribution of Reviewer and Tester. Attribution over the degraded peers correctly finds no intrinsic failure mode and returns `exogenous_disruption`—the empirical form of the attribution failure that Section VI-A predicts. Reintroducing the new agent under a scoped-context onboarding protocol (outputs quarantined from peer context during a probationary window, then released gradually) attenuates peer degradation by  $[A\%]$  relative to naive insertion.

#### D. Attribution Quality Across All Cells

Because every injected cell has ground truth, the attribution funnel’s output is scored as a confusion matrix (injected mode  $\times$  attributed mode) with per-mode recall and abstention rate—to our knowledge the first such accounting for a remediation protocol. Overall accuracy excluding abstentions is  $[Z]$  with an abstention rate of  $[W\%]$ ; we report the two together, since a stage that resolves every case by construction can achieve any accuracy demanded of it. Only APIP-condition runs enter the matrix: B2 and B3 do not attribute, and including B4 would render it perfectly diagonal by fiat.

#### E. CoffeeBench: Naturally Occurring Degradation

CoffeeBench places six agent-operated coffee firms in a persistent economic simulation. Its role is narrow and irreplaceable: with no injection at all, contract metrics drift as the world evolves, and the protocol must distinguish degradation warranting intervention from legitimate environmental change—the `recalibrated` exit outcome exists because of this environment. With  $[k]$  seeds (cost bounds replication), the no-injection scenario yields `[outcome]`; abstention or re-baselining here is a legitimate, reportable result, and economic collapse (bankruptcy) exercises `retired`.

#### F. Findings

Pending completion of the pre-registered runs, the analysis is structured around four questions: whether tier mismatch remains the dominant source of excess cost (the B2 dynamic, Section V-B); whether disaggregation materially shortens detection (Section V-C); whether token and latency movement—logged as contract *soft* metrics—leads quality breaches; and whether attribution is reliable enough to justify the mandatory attribution phase (Section V-D), the load-bearing question for the framework as a whole.

#### G. Limitations

Injection schedules in  $\tau^2$ -bench and ChatDev remain scripted; CoffeeBench is the sole source of emergent degradation and supports only  $[k]$  seeds. Two contract metrics depend on an LLM judge; we report judge-human agreement on a verified sample and enforce judge/backbone independence, but judge error remains a bounded unknown; the deployed metric judges one atomic claim at a time behind a recall-oriented pre-filter, and judge-scored metrics are baselined against the *current* policy version so triggers test divergence from the agent’s real error floor. Environment versions are pinned in every run manifest, and all runs, traces, and the frozen mapping table are released for replication.

### VI. OPEN PROBLEMS AND FUTURE DIRECTIONS

#### A. Multi-Agent Mesh Deployments: The New Hire Problem

The APIP as presented assumes one agent, one behavioral contract, one remediation process. In agent meshes—increasingly common in enterprise deployments—a failure mode emerges that is structurally invisible to this assumption: *mesh disruption*. A new specialist agent, Agent N, is



introduced into an existing mesh and performs its designated task well, yet aggregate task success or the performance of peer agents declines. The APIP triggers on the peers, but cause attribution finds no intrinsic failure mode—no drift, fresh retrieval, intact alignment. The root cause is exogenous, and a purely single-agent APIP has no mechanism to represent or address it. Kim et al. [26] provide direct empirical support, reporting performance change relative to single-agent baselines from +80.8% on decomposable tasks to −70.0% on sequential planning and identifying a coordination-saturation effect. Plausible mechanisms (shared context crowding, role boundary erosion, distributional shift in shared state, trust miscalibration toward the newcomer) all predict the same aggregate signature; our experiment (Section V-C) establishes that the externality exists, not which mechanism carries it—a formal treatment is reserved for a companion paper. The phenomenon is directly analogous to the organizational “new hire problem,” which Event System Theory [17] frames as an event whose impact scales with its novelty, disruptiveness, and criticality.

The framework already incorporates the two extensions this motivates: the `exogenous_disruption` failure mode, whose diagnosis keys on peer-role concentration coincident with a mesh change, and the scoped-context onboarding protocol evaluated in Section V-C—context scoping during a probationary window with regression monitoring on peers, analogous to structured newcomer onboarding [18]. Scale remains open: whether attribution stays tractable as meshes grow to hundreds of agents exceeds current tooling [27].

### B. Open Problems

Three problems remain open. *Behavioral contract specification*: contracts are architecturally foundational but absent from most deployments, and authoring one requires prospective agreement across ML engineering, product, and legal, with no supporting tooling yet. *Cause attribution at scale*: the human adjudication step (Section IV-B) does not scale; automated methods reach only 36.2% step-level accuracy [20], [21], and they attribute individual trajectories where the APIP needs window-level classification—the gap the funnel of Section IV-B1 is a first attempt to close. *Organizational ownership*: in HR the PIP is jointly owned by manager and HR function; in AI deployments the equivalent is unresolved, and without an ownership model adoption will stall.

## VII. CONCLUSION

We have introduced the APIP, a structured post-deployment remediation protocol adapting the human PIP’s lifecycle primitives—trigger, cause attribution, tiered intervention, bounded window, exit evaluation—with Kirkpatrick’s four-level model distinguishing durable recovery from superficial improvement. Across three live environments the protocol is scored end-to-end against injected ground truth; the harness, run artifacts, version pins, and frozen mapping table are released so its pre-registration claims are checkable. Our central argument is simple: deployed agents should be treated

as subjects of ongoing accountability rather than static shipped artifacts—a discipline the AI field can adopt deliberately, before regulation makes it mandatory.

## REFERENCES

- [1] P. Liang *et al.*, “Holistic evaluation of language models,” arXiv preprint arXiv:2211.09110, 2022.
- [2] D. Ganguli *et al.*, “Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned,” arXiv preprint arXiv:2209.07858, 2022.
- [3] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [4] M. Z. Pan, N. Arabzadeh, and M. Ellis, “Measuring agents in production,” arXiv preprint arXiv:2512.04123, 2025.
- [5] European Parliament and Council, “Regulation on artificial intelligence (EU AI Act),” *Official Journal of the European Union*, 2024.
- [6] S. Wachter, B. Mittelstadt, and C. Russell, “Counterfactual explanations without opening the black box: Automated decisions and the GDPR,” *Harvard Journal of Law & Technology*, vol. 31, no. 2, 2017.
- [7] D. L. Kirkpatrick, *Evaluating Training Programs: The Four Levels*. San Francisco, CA, USA: Berrett-Koehler, 1994.
- [8] M. Cemri, M. Z. Pan, S. Yang *et al.*, “Why do multi-agent LLM systems fail?” arXiv preprint arXiv:2503.13657, 2025.
- [9] G. Sapra *et al.*, “AgentCompass: Towards reliable evaluation of agentic workflows in production,” arXiv preprint arXiv:2509.14647, 2025.
- [10] A. Rath, “Agent drift: Quantifying behavioral degradation in multi-agent LLM systems,” arXiv preprint arXiv:2601.04170, 2026.
- [11] K. Zhu, X. Ye, Z. Han *et al.*, “AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents,” arXiv preprint arXiv:2607.18754, 2026.
- [12] J. D. Kirkpatrick and W. K. Kirkpatrick, *Kirkpatrick’s Four Levels of Training Evaluation*. Association for Talent Development, 2016.
- [13] N. Kolt, L. Heim, and M. Anderljung, “Visibility into AI agents,” in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2024.
- [14] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, “AI control: Improving safety despite intentional subversion,” arXiv preprint arXiv:2312.06942, 2023.
- [15] A. Bhatt *et al.*, “Ctrl-Z: Controlling AI agents via resampling,” arXiv preprint arXiv:2504.10374, 2025.
- [16] B. Schoen *et al.*, “Stress-testing control protocols with adaptive attacks against trusted monitors,” arXiv preprint arXiv:2510.05367, 2025.
- [17] F. P. Morgeson, T. R. Mitchell, and D. Liu, “Event system theory: An event-oriented approach to the organizational sciences,” *Academy of Management Review*, vol. 40, no. 4, pp. 515–537, 2015.
- [18] J. D. Kammeyer-Mueller, C. R. Wanberg, A. Rubenstein, and Z. Song, “Support, undermining, and newcomer socialization: Fitting in during the first 90 days,” *Academy of Management Journal*, vol. 56, pp. 1104–1124, 2013.
- [19] D. Hendrycks *et al.*, “Aligning AI with shared human values,” arXiv preprint arXiv:2008.02275, 2021.
- [20] Y. Ma *et al.*, “Automatic failure attribution and critical step prediction for multi-agent systems based on causal inference,” arXiv preprint arXiv:2509.08682, 2025.
- [21] S. Zhang *et al.*, “Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems,” in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025, arXiv:2505.00212.
- [22] “AgentTrace: Causal graph tracing for root cause analysis in multi-agent systems,” arXiv preprint arXiv:2603.14688, 2026.
- [23] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan, “ $\tau^2$ -bench: Evaluating conversational agents in a dual-control environment,” arXiv preprint arXiv:2506.07982, 2025.
- [24] C. Qian, W. Liu, H. Liu *et al.*, “ChatDev: Communicative agents for software development,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
- [25] Sakana AI, “CoffeeBench: A persistent economic simulation benchmark for LLM agents,” <https://github.com/SakanaAI/CoffeeBench>, 2026.
- [26] Y. Kim *et al.*, “Towards a science of scaling agent systems,” arXiv preprint arXiv:2512.08296, 2025.
- [27] N. Krishnan, “Advancing multi-agent systems through model context protocol,” arXiv preprint arXiv:2504.21030, 2025.